PalmSource™ 99

# Designing for Globalization

Chris Schneider
TransPac Software, Inc.

3Com
More connected.

# What's this Talk About?

PalmSource 99

- Showing how easy it is to expand into the international market
- Avoiding common mistakes that complicate globalization

# Topics Covered

PalmSource 99

- Overlay resource databases
- Text manager
- Date and time manager
- Top 10 mistakes complicating globalization
- Whatever I forget to say (Q & A)

# Overlay Resource Databases

PalmSource 99

- Separate resource databases containing localized resources
- One-to-one mapping from base resource DB to overlay resource DB
  - Any app, panel, extension or library can be overlaid
- Overlay manager automatically substitutes localized resources at runtime
  - No need to ever call overlay manager
  - If overlay resource DB exists, localized resources are returned by data manager

# Text Manager Support

PalmSource 99

- Character sizes, attributes and encodings
- Word bounds and string truncation
- Searching, comparing and transliterating text

# Date and Time Manager Support

PalmSource 99

- Format of textual representations depends on system locale
- New support for custom date and time formats

# Top 10 Mistakes Complicating Globalization

PalmSource 99

- Your handout contains a more detailed list with both good and bad example code
- Sample code in slides shows correct method
- Make notes on items you think might apply to your software
- Try to save questions for the end of the talk

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#10 Forgetting to Check a keyDown's Command Bit*

- Make sure you've got a virtual character:
  - UNICODE encodings will probably overlap the virtual character codes someday

```
if (TxtCharIsHardKey(
   e->data.keyDown.modifiers,
   e->data.keyDown.chr) {
   return(false);
} else if (  e->data.keyDown.modifiers
           & commandKeyMask) {
   // handle chrPageUp, etc.
} else {
   // handle printable characters...
}
```

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#9 Using High-ASCII Characters*

- Ellipses, curly quotes, and numeric spaces can cause cosmetic problems
- Either determine the correct glyph to use at runtime or just use simpler characters
  - chrEllipsis and chrNumericSpace moved to control range, thus OK for all fonts on new ROMs
- Use PalmOSGlue.lib if you need to run on older ROMs
  - The glue (e.g. ChrNumericSpace) calls the IntlDispatch trap if present, otherwise it provides the basic Latin support

# Top 10 Mistakes Complicating Globalization

PalmSource 99

*#8 Leaving No Room For Translations*

- Localization can increase field width by 50%
- Longer text can also overflow short string buffers in code and database fields

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#7 Calling Old Character Attribute Macros*

- GetCharAttr won't work for multi-byte characters
- The new calls are easier anyway:

```
if (TxtCharIsPrint(e->data.keyDown.chr))
{
   // handle printable characters...
}
```

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#6 Treating Bytes as Characters*

- Replace all char and Char declarations with WChar declarations
  - Use Byte for non-textual data
- Use TxtGetNextChar, TxtCharWidth, etc:

```
while (width > maxWidth) {
   WChar theWChar;
   len -= TxtGetPreviousChar(  str,
                               len,
                               &theWChar);
   width -= TxtCharWidth(theWChar);
}
```

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#5 Truncating Text by Hand*

- Use TxtGetTruncationOffset to limit buffer lengths without splitting multi-byte characters:

```
buffer[TxtGetTruncationOffset(buffer,
                              kMaxLen)]
   = 0;
```

- Use WindDrawTruncChars or FntWidthToOffset to limit field widths:

```
len = FntWidthToOffset(str, len, width -
   TxtCharWidth(chrEllipsis), NULL, NULL);
len += TxtSetNextChar(str, len,
   chrEllipsis);
str[len] = 0;
```

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#4 Transliterating Text by Hand*

- Your home-grown upper-case function probably doesn't even handle European characters, let alone Japanese ones
- StrToLower is very limited
  - Depending on ROM, it may only handle A…Z
- Macros like TxtUpperStr make using TxtTransliterate trivial
  - Again, use PalmOSGlue.lib if you need to run on old ROMs

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#3 Calling FindStrInStr*

- Length of found text is no longer guaranteed to match length of search string, so use TxtFindString & stash length in custom parm:

```
if (TxtFindString(stringFromAppData,
   fParms->strToFind, &pos, &matchLen)) {
   done = FindSaveMatch(fParms,
      recordNum, pos, fieldNum, matchLen,
      cardNo, dbID);

...then, in response to a frmGotoEvent...

event.data.frmGoto.matchLen
   = gParms->matchCustom;
```

# Top 10 Mistakes Complicating Globalization

PalmSource 99

*#2 Concatenating Sub-strings by Hand*

- Don’t make assumptions about order of parameters in a string
- TxtReplaceStr supports much more flexible template strings
- TxtParamString allocates a string for you and fills it using a template string resource and up to 4 parameter strings

# Top 10 Mistakes Complicating Globalization

PalmSource 99

## *#1: Hard-coding Text and Other Data*

- Interface data belongs in resources, period.
  - Use standard resource types, since custom types confuse both localizers and localization tools
- Don't forget numeric constants you might be using for sizes, widths, counts, etc:

```
UInt32 ResLoadConstant(UInt16 rscID);
```

# Summary

PalmSource 99

- Let Palm OS® software handle your locale and ROM dependencies for you
- Move all localizable data into resources
- Leave room for expanded localizations
- Track all changes to resources


PalmSource™ 99
3Com
More connected.